PROGRESSION · TIMELINE · STRUCTURE

list-steps

Vertical sequence of steps, each with full description body.

How to add a new component to Lattice.

STEP 01

Scaffold the folder

Create `lib/components/<name>/` with a manifest declaring name, function, form, substance, slots, and skeleton.

STEP 02

Author the styles

Scope the CSS to the section class. Use palette tokens — no hex literals in layout rules.

STEP 03

Add a transform if needed

Substance `structure` or `series` ships a transform module wired into all three render paths.

STEP 04

Demo and document

Author `<name>.example.md` and enrich the manifest with sample, whenToUse, antiPatterns, and related. The generator emits the docs and gallery sidecars.

Three phases, vertically arranged.

STEP 01

Discover

Interview eight stakeholders. Open questions only — listening for friction, not confirming assumptions.

STEP 02

Frame

Half-day workshop to align on root cause. Output is a ranked problem statement.

STEP 03

Decide

Written sign-off on what is in scope, what is out, and what requires a separate decision.

A four-phase engagement model.

PHASE 01

Discovery

Eight weeks. Stakeholder interviews, current-state audit, and a problem-framing workshop produce a signed scope.

PHASE 02

Design

Six weeks. Two design partners co-build the operating model and the change-management plan.

PHASE 03

Pilot

Twelve weeks. One business unit runs the model end-to-end with weekly retrospectives.

PHASE 04

Rollout

Phased by region. Pilot learnings shape the rollout cadence; central team owns the playbook.

Three milestones to GA.

MILESTONE A

Closed beta

Five design-partner accounts live on the platform. Daily standups; weekly retros.

MILESTONE B

Open beta

Self-serve signup at the marketing site. Pricing visible but not enforced.

MILESTONE C

GA

Billing enforcement on. SLA enters effect. Support escalation paths published.

Three tracks for the next quarter.

STEP A

Platform hardening

Multi-tenant DEKs, automated rotation, and the crypto-shred runbook land in this track.

STEP B

Compliance posture

Examiner pack v2 and the centralised audit log ship for the Q3 audit window.

STEP C

Developer surface

Polyglot SDK parity and the new CLI flags close out the API roadmap.

Three stages, with explicit stage prefixes.

STAGE 01

Plan

Define the work and the artifacts each stage produces.

STAGE 02

Execute

Run the work against the plan; track variance.

STAGE 03

Review

Compare actuals to plan; capture lessons for the next cycle.

Top three risks, ranked by exposure.

RANK 01

Renewal cohort

$2.1M ARR at risk if pricing comp gap persists.

RANK 02

Pipeline conversion

11 pp below Q1; legal review is the chokepoint.

RANK 03

Competitive displacement

Seven losses to one competitor in the $80-200K tier.

Three engagement tiers.

TIER I

Strategic

Quarterly executive review, dedicated success manager.

TIER II

Growth

Monthly check-in, shared success pool.

TIER III

Self-serve

Async docs, community support.

Three phases, roman-numeral counter.

PHASE I

Discovery

Identify the constraints and the success criteria.

PHASE II

Design

Sketch options against the constraints; pick one.

PHASE III

Delivery

Build, ship, measure.

How to add a new component to Lattice.

STEP 01

Scaffold the folder

Create `lib/components/<name>/` with a manifest declaring name, function, form, substance, slots, and skeleton.

STEP 02

Author the styles

Scope the CSS to the section class. Use palette tokens — no hex literals in layout rules.

STEP 03

Add a transform if needed

Substance `structure` or `series` ships a transform module wired into all three render paths.

STEP 04

Demo and document

Author `<name>.example.md` and enrich the manifest with sample, whenToUse, antiPatterns, and related. The generator emits the docs and gallery sidecars.

How to add a new component to Lattice.

STEP 01

Scaffold the folder

Create `lib/components/<name>/` with a manifest declaring name, function, form, substance, slots, and skeleton.

STEP 02

Author the styles

Scope the CSS to the section class. Use palette tokens — no hex literals in layout rules.

STEP 03

Add a transform if needed

Substance `structure` or `series` ships a transform module wired into all three render paths.

STEP 04

Demo and document

Author `<name>.example.md` and enrich the manifest with sample, whenToUse, antiPatterns, and related. The generator emits the docs and gallery sidecars.

How to add a new component to Lattice.

STEP 01

Scaffold the folder

Create `lib/components/<name>/` with a manifest declaring name, function, form, substance, slots, and skeleton.

STEP 02

Author the styles

Scope the CSS to the section class. Use palette tokens — no hex literals in layout rules.

STEP 03

Add a transform if needed

Substance `structure` or `series` ships a transform module wired into all three render paths.

STEP 04

Demo and document

Author `<name>.example.md` and enrich the manifest with sample, whenToUse, antiPatterns, and related. The generator emits the docs and gallery sidecars.

When NOT to reach for list-steps.

Light labels, no body. If each step is a single label with no description, use `timeline`. list-steps earns its chrome only when the body adds substance.

Parallel options. If the rows are alternatives the audience compares, use `cards-grid` or `verdict-grid`. The numbered prefix here reads as sequence — using it for parallel items mis-cues the audience.

Author-typed step numbers. Don't write `<strong>STEP 01</strong>` into the markdown. The badge is CSS-generated from the `ol` counter; manual numbering double-stamps and breaks on reordering.

See also.

RELATED COMPONENTS

`timeline` — shorter labels per step, horizontal axis instead of vertical cards

`list-criteria` — gating requirements rather than a sequence of actions

`split-steps` — phase label + heading on the left, steps on the right

`roadmap` — phased grid across multiple workstreams

`principles` — tenets or values rather than a procedural sequence